TASK 05 – READ INPUT AND CONVERT TYPES

Assignment: Design, Implement and Validate a Payment Calculator for a Village Milk Co-operative
Bloom's Level: K3 – Apply
Marks: 3

1. Aim

To read milk details from the clerk, convert the input into suitable numeric data types, handle invalid entries and display the payment details using formatted output.

2. Why Does input() Return a String?

In Python, the input() function always returns the entered value as a string, even when the user enters a number.

Example:

```
age = input("Enter age: ")

print(type(age))
```

If the user enters 20, the output is:

<class 'str'>

To perform mathematical calculations, we must convert the input into a numeric type.

- int() converts a value into an integer.
- float() converts a value into a decimal number.

Example:

```
litres = int("10")

fat = float("4.2")
```

3. Handling Invalid Input

The clerk may enter incorrect values.

Input	Result
8.5	Valid decimal number
8.5	Valid after removing surrounding spaces
8,5	Invalid for Python's float()
eight	Invalid because it is not numeric

If we try to convert invalid text using float(), Python raises a ValueError.

Example:

fat = float("eight")

This produces a ValueError.

The program should handle this error instead of stopping unexpectedly.

4. Checking and Converting Input

One approach is to use string methods such as strip() to remove unwanted spaces and then convert the value.

Another approach is to use try/except ValueError. The program attempts the conversion and displays an error message if the input is invalid.

Example:

```
text = input("Enter fat percentage: ")


try:
    fat = float(text.strip())
    print("Valid fat:", fat)
except ValueError:
    print("Invalid input")
```

This approach accepts 8.5 and 8.5 but rejects 8,5 and eight.

5. Formatted Output Using f-Strings

Python f-strings help display values neatly.

Example:

```
amount = 300
print(f"Amount: Rs. {amount:8.2f}")
```

Output:

```
Amount: Rs.   300.00
```

Here:

- 8 specifies the minimum field width.
- .2f displays exactly two digits after the decimal point.

This is useful for displaying milk rates and farmer payments in a clear format.

6. Alternative Concept and Complexity Analysis

Instead of checking the input using string methods before conversion, we can directly use try/except ValueError.

Example:

```
text = input("Enter litres: ")


try:
    litres = float(text.strip())
    print("Valid:", litres)
except ValueError:
    print("Invalid number")
```

Comparison:

Input	Result using try/except
8,5	Rejected
8.5	Accepted
eight	Rejected

Complexity analysis:

- Both approaches require time proportional to the length of the input string, O(m), where m is the number of characters.
- Extra space is O(1), excluding the input string itself.
- try/except is convenient because it directly handles conversion errors.
- String checking can help detect certain incorrect formats before conversion, but it does not replace numeric conversion.

7. Result

The input process was designed to accept milk details as strings, convert them into numeric values, handle invalid entries and display payment information using formatted f-strings. The try/except method provides a simple way to prevent invalid input from stopping the program.